# LESSON 27

# 一个一个 for 试

## 枚举

学完以后，遇到“哪些数满足条件”的题，会先想清楚从哪试到哪，再让电脑一个一个试。

# # 12 能被哪些数整除?

我们可能会心算：1、2、3、4、6、12。可要是 n 是 840 呢？心算容易漏。

电脑不怕累：从 1 试到 n，每个都问一句“n 能被它整除吗”，能就记下来。

把所有可能一个一个试一遍，叫**枚举**。
要先想清楚两件事：**从哪试到哪**，**满足什么条件**。

# 你来当计算机，试一试

try.cpp

```cpp
#include <iostream>
using namespace std;
int main() {
    int i = 5;
    cout << 12 % i << endl;
    cout << (12 % i == 0) << endl;
    return 0;
}
```

第 5 行：12 除以 i 余几

**2**

第 6 行：能整除吗

**0**

从 1 改到 12，哪几个得 1？

你一个个试，就是在做枚举。程序替你试，快得多。

# 从 1 试到 n，能整除就输出

divisors.cpp

```
7       for (int i = 1; i <= n; i++) {
8           if (n % i == 0) {
9               cout << i << " ";
10          }
11      }
12      cout << endl;
```

输入 12

**1 2 3 4 6 12**

从哪试到哪：for 写的是 1 到 n。

满足什么条件：if 写的是 n % i == 0，余数是 0 就是整除。

前面先读入 n，n 是 1 到 1000 的整数。

# n 是 12：试了 12 次，6 次成功




实线涂色的是 `12 % i == 0` 成立的，虚线的是不成立的。

枚举就是这样：每一个都试，不跳过，所以不会漏。
动手前先估一估要试几次：这里 n 是几就试几次，几千次电脑一下就试完。要试的次数太多，就得想更快的办法。

# # 小测：n 是 7，n % i == 0 成立了几次?

i 从 1 试到 7，成立一次就 count++，最后输出 count。

1 7

2 2

3 1

4 0

答对了！只有 1 和 7 能整除 7。

# 猜一猜：<= 写成了 <

divisors.cpp（只阅读）

```
7       for (int i = 1; i < n; i++) {
```

屏幕上

**1 2 3 4 6**

别的都不变，还是输入 12，屏幕上是什么？

少了 12：i 只试到 11，n 自己没试。
写枚举先问自己：最小的和最大的那个，试到了吗？

# 动手改一改

从 divisors 开始。不用输出因数，数一数有几个。

## 是不是质数?

读入一个正整数 n。正好有 2 个因数（1 和它自己）的叫质数。是质数输出 Yes，不是输出 No。

| 输入 7 | 输入 9 | 输入 1 |
| --- | --- | --- |
| Yes | No | No |

# 鸡兔同笼：试鸡有几只

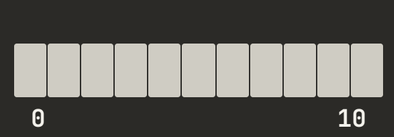
chicken.cpp

```cpp
    for (int c = 0; c <= heads; c++) {
        int r = heads - c;
        if (2 * c + 4 * r == legs) {
            cout << c << " " << r << endl;
            found = true;
        }
    }
```

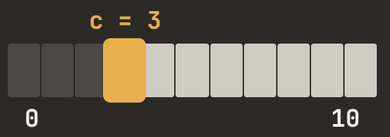
10 个头，32 只脚。鸡可能有 0 到 heads 只，鸡定了，兔子就是 heads - c。

鸡 2 只脚，兔 4 只脚，加起来正好等于 legs 就对上了。

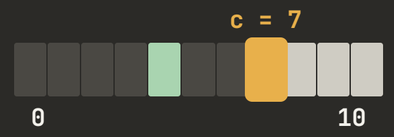
found 和第 13 课一样，记一下找到过没有。



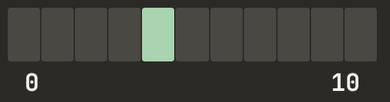

**第 1 步** c 从 0 试到 10，一个一个试。




**第 5 步** c 是 3：算出 34，不是 32，试下一个。




**第 9 步** c 是 7：算出 26，不是 32，试下一个。




**第 13 步** 试完了，一共试了 11 个，对上的只有 4。

# 猜一猜：3 个头，7 只脚

chicken.cpp

```
15      if (!found) {
16          cout << "No answer" << endl;
17      }
```

屏幕上

**No answer**

c 从 0 试到 3，脚数分别是 12、10、8、6。有没有 7？

每一种都试过了，都不对，就是真的没有。

枚举的好处：试完全部还没找到，就能放心地说“没有”。

# 动手改一改

从 chicken 开始。这回试 5 元有几张，剩下的给 2 元。

## 凑钱有几种?

只有 2 元和 5 元两种钱，张数不限，可以不用某一种。读入 n，输出正好凑出 n 元有几种凑法。

| 输入 10 | 输入 1 | 输入 0 |
| --- | --- | --- |
| **2** | **0** | **1** |

# 找错：7 怎么只有 1 个因数？

prime.cpp（只阅读）

```
1       int count = 0;
2       for (int i = 1; i < n; i++) {
3           if (n % i == 0) {
4               count++;
5           }
6       }
```

点代码里的一行

这段想数出 n 有几个因数，最后输出 count，这里没画。

输入 7，应该是 2，屏幕上却是 1。

点你认为错了的那一行。

# 自己挑战

# 各位加起来是 k

一行读入两个正整数 n 和 k（n 不超过 10000，k 不超过 36）。1 到 n 里，有几个数的各位数字加起来正好是 k？一个也没有就输出 0。比如 21 的各位加起来是 2 + 1 = 3。

输入 30 3，输出

```
4
```

输入 100 1，输出

```
3
```

输入 9 10，输出

```
0
```

先自己写，卡住了再翻下一页。

# 卡住了？一次看一条提示

**提示 1**　从哪试到哪：i 从 1 到 n。满足什么条件：i 的各位加起来是 k。

**提示 2**　拆数位第 7 课倒着输出数字时用过：x % 10 是个位，x / 10 去掉个位，x 变 0 就停。

**提示 3**　不要直接拆 i，先 int x = i; 再拆 x，不然 for 的 i 被改坏了。

# # 密码有几种?

自己定一套规则，让电脑把所有可能都试一遍，数出符合规则的有几种。主题你自己定。

### 先做到

说清从哪试到哪、满足什么条件，再输出有几种。

### 选你喜欢的

三位密码锁、分糖果、排队站位、买文具凑钱，都可以。

### 还想更厉害

把符合的全都列出来，或者让用户输入规则里的数。

没有标准答案。写之前先手算一个小的例子，对一对结果。

# # 和同桌换着玩

## 1

### 换着运行

运行同桌的程序，先听规则，再自己手算一个小的对一对

## 2

### 说一个喜欢的

告诉同桌，你最喜欢这个程序哪里

## 3

### 提一个想法

问问同桌：最小和最大的那种，试到了吗？

听完再改一版。想一想、做出来、试一试、分享、再改，程序就是这样越做越好的。

# 说得清吗?

**Q1** **写枚举前要先想清楚哪两件事?**

从哪试到哪，满足什么条件。

**Q2** **鸡兔同笼为什么只试鸡，不用再套一层试兔子?**

鸡定了，兔子就是 `heads - c`，能算出来的就不用试。

**Q3** **试完全部都没找到，说明什么?**

说明真的没有，可以放心输出“没有”。

# 速查卡

| | |
|---|---|
| `i <= n` | 从哪试到哪，两头都要试到 |
| `if (条件)` | 满足就输出，或者 count++ |
| `r = heads - c` | 能算出来的，就不用再套一层去试 |
| `bool found` | 试完都没找到，就输出“没有” |

# # 这些后面再学

## 递推

下一个由前一个算出来，不用每次从头试

## 试得更少

比如找因数只试到根号 n，数大了才需要

今天学会枚举：想清楚范围和条件，让电脑一个一个试。

# 谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 cpp-for-youth

下一课：递推